

[Bài đọc] Spring Framework là gì? Giới thiệu IOC, DI

1. Spring Framework là gì?

Spring Framework là một framework mã nguồn mở (open-source) được viết bằng ngôn ngữ lập trình Java, được thiết kế để đơn giản hóa việc phát triển ứng dụng doanh nghiệp (enterprise applications). Spring cung cấp một nền tảng toàn diện để xây dựng các ứng dụng mạnh mẽ, linh hoạt và dễ bảo trì. Nó được sử dụng rộng rãi trong các ứng dụng web, ứng dụng doanh nghiệp, và các hệ thống phân tán.

Spring Framework hỗ trợ các nhà phát triển bằng cách cung cấp các công cụ và tính năng giúp giảm độ phức tạp khi viết mã, chẳng hạn như:

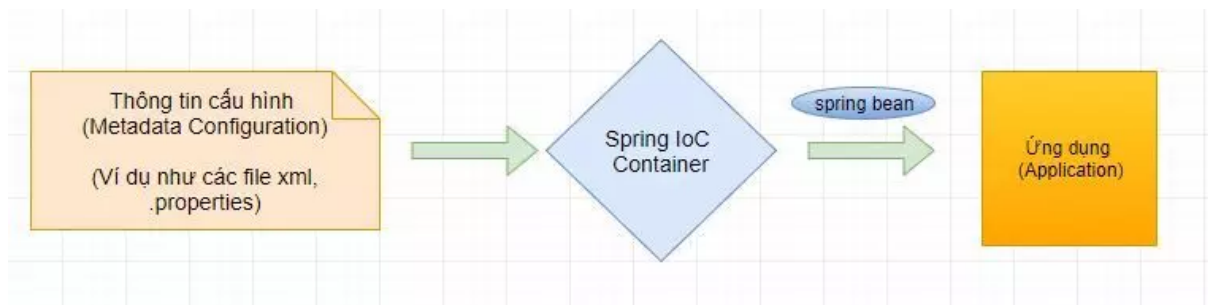
- **Quản lý phụ thuộc (Dependency Management):** Giúp quản lý các thành phần của ứng dụng một cách dễ dàng.
- **Hỗ trợ lập trình hướng khía cạnh (Aspect-Oriented Programming - AOP):** Tách biệt các logic phụ trợ (như logging, bảo mật) khỏi logic chính.
- **Tích hợp dễ dàng:** Hỗ trợ tích hợp với các công nghệ khác như Hibernate, JPA, RESTful APIs, và các cơ sở dữ liệu.
- **Mô hình MVC:** Spring MVC cung cấp cấu trúc để xây dựng các ứng dụng web một cách có tổ chức.
- **Hỗ trợ giao dịch (Transaction Management):** Đơn giản hóa việc quản lý giao dịch trong cơ sở dữ liệu.



Spring Framework được thiết kế theo nguyên tắc "**Convention over Configuration**" (ưu tiên quy ước hơn cấu hình) và "**Don't Repeat Yourself**" (DRY), giúp giảm thiểu mã lặp và tăng hiệu quả phát triển.

2. IoC (Inversion of Control - Đảo ngược điều khiển)

IoC là một nguyên tắc thiết kế trong lập trình hướng đối tượng, được Spring Framework sử dụng để quản lý các thành phần (components) của ứng dụng. Thay vì để các đối tượng tự khởi tạo và quản lý các phụ thuộc của chúng, IoC chuyển trách nhiệm này cho một **container** (thường gọi là Spring IoC Container).



2.1. Cách IoC hoạt động

- Trong mô hình truyền thống, một đối tượng (object) tự chịu trách nhiệm tạo và quản lý các phụ thuộc của nó. Ví dụ, nếu một lớp UserService cần sử dụng UserRepository, nó sẽ tự tạo đối tượng UserRepository bằng từ khóa new.
- Với IoC, Spring Container (thường là ApplicationContext) chịu trách nhiệm tạo, cấu hình, và quản lý vòng đời của các đối tượng (gọi là **beans**). Các đối tượng không cần tự tạo phụ thuộc, mà Spring sẽ "tiêm" (inject) các phụ thuộc này vào chúng.

2.2. Lợi ích của IoC

- **Giảm sự phụ thuộc chặt chẽ (Loose Coupling):** Các lớp không cần biết chi tiết về cách tạo các phụ thuộc, giúp mã dễ bảo trì và mở rộng hơn.
- **Dễ dàng kiểm thử (Testability):** Vì phụ thuộc được tiêm từ bên ngoài, việc thay thế các đối tượng giả lập (mock objects) trong kiểm thử đơn vị (unit testing) trở nên dễ dàng.
- **Quản lý tập trung:** Spring Container quản lý tất cả các bean, giúp dễ dàng cấu hình và theo dõi ứng dụng.
- **Tính linh hoạt:** Dễ dàng thay đổi hoặc cấu hình lại các phụ thuộc mà không cần sửa đổi mã nguồn.

3. DI (Dependency Injection - Tiêm phụ thuộc)

DI là một cách triển khai cụ thể của IoC, trong đó các phụ thuộc của một đối tượng được "tiêm" từ bên ngoài thay vì tự tạo bên trong. Trong Spring, DI là cơ chế chính để hiện thực hóa IoC.

3.1. Các hình thức DI trong Spring

Spring hỗ trợ ba cách chính để thực hiện DI:

1. **Constructor Injection** (Tiêm qua hàm khởi tạo): Phụ thuộc được cung cấp thông qua constructor của lớp.

2. **Setter Injection** (Tiêm qua phương thức setter): Phụ thuộc được tiêm thông qua các phương thức setter.
3. **Field Injection** (Tiêm trực tiếp vào thuộc tính): Phụ thuộc được tiêm trực tiếp vào các trường (field) bằng annotation `@Autowired`. (Lưu ý: Cách này ít được khuyến khích vì khó kiểm thử và kiểm soát).

3.2. Lợi ích của DI

- **Tính mô-đun:** Các thành phần của ứng dụng được tổ chức độc lập, dễ dàng thay thế hoặc nâng cấp.
- **Dễ kiểm thử:** DI cho phép thay thế các phụ thuộc bằng các mock objects trong quá trình kiểm thử.
- **Tái sử dụng mã:** Các bean được quản lý bởi Spring có thể được tái sử dụng trong nhiều phần của ứng dụng.
- **Cấu hình linh hoạt:** Có thể cấu hình phụ thuộc thông qua XML, Java Config, hoặc annotation, giúp dễ dàng thay đổi hành vi của ứng dụng.